# COMPREHENSIVE GUIDE TO RAG

## Retrieval-Augmented Generation

Theory • Implementation • Applications • LLMs

Created: May 27, 2026

# TABLE OF CONTENTS

# 1. WHAT IS RAG?

**RAG (Retrieval-Augmented Generation)** is a technique that combines information retrieval with text generation using Large Language Models (LLMs). Instead of relying solely on the knowledge encoded in an LLM's weights, RAG retrieves relevant documents/context from an external knowledge base and uses them to inform the generation process.

**Simple Definition:** Given a query, RAG retrieves relevant documents from a corpus and uses both the query and retrieved documents to generate accurate, grounded answers.

**Key Insight:** "Provide the model with the information it needs to answer correctly rather than expecting it to remember everything."

## 1.1 Basic RAG Flow

1. **User Query:** "What are the benefits of renewable energy?"
2. **Retrieval:** Find relevant documents (using semantic search)
3. **Ranking:** Rank retrieved documents by relevance
4. **Context Preparation:** Create prompt with query + retrieved documents
5. **Generation:** LLM generates answer based on context
6. **Response:** Return generated answer to user

## 1.2 Why RAG Matters

- **Reduces Hallucinations:** Grounds answers in retrieved documents
- **Up-to-date Information:** Can include recent documents without retraining LLM
- **Fact Verification:** Answers can be traced back to sources
- **Domain-Specific Knowledge:** Works with proprietary/custom documents
- **Cost Efficient:** Smaller models + retrieval can outperform large models
- **Transparency:** Users see which documents informed the answer

# 2. PROBLEM RAG SOLVES

## 2.1 LLM Limitations

**Problem 1: Knowledge Cutoff**
LLMs are trained on data up to a specific date. GPT-3.5 was trained until April 2023. They cannot answer questions about events after training.

**Problem 2: Hallucinations**
LLMs generate plausible-sounding but false information. Example: Inventing citations, making up facts, confusing details.

**Problem 3: Proprietary/Private Data**
LLMs cannot access internal documents, databases, or proprietary information. Retraining is expensive and time-consuming.

**Problem 4: Context Window Limitations**
Even with 100K token windows, LLMs can't efficiently process all available documents. Need to select most relevant ones.

**Problem 5: Traceability**
Hard to verify where LLM got information. No sources = hard to trust or verify claims.

## 2.2 RAG Solutions

✓ **Knowledge Cutoff:** Retrieve latest documents from knowledge base
✓ **Hallucinations:** Ground answers in retrieved facts
✓ **Proprietary Data:** Add private documents to retrieval system
✓ **Context Window:** Smart retrieval selects most relevant subset
✓ **Traceability:** Quote sources, provide document references

**Result:** More accurate, reliable, trustworthy answers with sources

# 3. RAG ARCHITECTURE & COMPONENTS

## 3.1 Core Components

**1. Document Store / Knowledge Base**
- Collection of documents (PDFs, web pages, databases, etc.)
- Needs to be chunked into smaller, retrievable units
- Can be: Vector DB, relational DB, document index

**2. Embedding Model**
- Converts text to vectors (embeddings)
- Examples: sentence-transformers, OpenAI embeddings, Cohere
- Must be semantic-aware (similar meaning = similar vectors)

**3. Retrieval System**
- Finds relevant documents given a query
- Uses semantic similarity (cosine, dot product)
- Can also use BM25, dense+sparse hybrid retrieval

**4. Ranking/Reranking**
- Orders retrieved documents by relevance
- Can use cross-encoders for more accurate ranking
- Improves quality of selected context

**5. LLM (Large Language Model)**
- Generates final answer
- Takes: query + retrieved context
- Examples: GPT-4, Claude, Llama-2

**6. Prompt Engineering**
- Structures how context is presented to LLM
- Instructions for format, sources, verification
- Critical for quality outputs

## 3.2 RAG Architecture Diagram

USER QUERY ↓ EMBEDDING MODEL (Convert query to vector) ↓ RETRIEVAL SYSTEM (Find similar documents) ↓ RETRIEVED DOCUMENTS (Top K results) ↓ RANKING/RERANKING (Order by relevance) ↓ CONTEXT PREPARATION (Format for LLM) ↓ PROMPT CONSTRUCTION (Query + Context + Instructions) ↓ LLM GENERATION (Generate answer) ↓ POST-PROCESSING (Format, citations, verification) ↓ FINAL RESPONSE TO USER

## 3.3 Indexing Phase

Happens ONCE during setup:

1. **Document Collection:** Gather all documents
2. **Document Cleaning:** Remove noise, standardize format
3. **Chunking:** Split into overlapping chunks (256-1024 tokens)
4. **Embedding:** Convert chunks to vectors using embedding model
5. **Storage:** Store vectors in vector database with metadata
6. **Indexing:** Create search indexes for fast retrieval

# 4. RETRIEVAL SYSTEMS

## 4.1 Retrieval Methods

**1. Semantic Retrieval (Dense Retrieval)**
- Uses embeddings and vector similarity
- Query → Embedding → Similarity search → Top K documents
- Pros: Semantic understanding, fast with indexes
- Cons: Needs good embedding model, no exact matching
- Example: Vector database search

**2. Lexical Retrieval (Sparse Retrieval)**
- BM25, TF-IDF algorithms
- Exact keyword matching with statistics
- Pros: Exact match, no training needed
- Cons: Misses semantic similarity, struggles with synonyms
- Example: Traditional search engines

**3. Hybrid Retrieval**
- Combines semantic + lexical search
- Takes union/intersection of both results
- Pros: Best of both worlds, more robust
- Cons: Slightly slower, more complex to tune
- Example: Elasticsearch + semantic search

## 4.2 Advanced Retrieval Techniques

**Multi-Query Retrieval:**
- Generate multiple query variations
- Retrieve with each variation
- Combine results
- Improves recall (finds more relevant docs)

**Recursive Retrieval:**
- First retrieval returns summary nodes
- Second retrieval gets detailed content
- Hierarchical document structure

**ColBERT (Contextualized Late Interaction):**
- Token-level embeddings
- Scores interactions between query and document
- Very accurate but slower

**Query Expansion:**
- Add related terms to query
- Use word embeddings or LLM to generate synonyms
- Broader retrieval coverage

# 5. VECTOR DATABASES & EMBEDDINGS

## 5.1 Vector Databases

**Popular Vector DBs:**

**1. FAISS (Facebook AI Similarity Search)**
- Open source, CPU/GPU support
- Good for research, small-to-medium scale
- Indexes: Flat, IVF, HNSW

**2. Pinecone**
- Managed cloud vector database
- Easy setup, built-in reranking
- Pay-as-you-go pricing

**3. Weaviate**
- Open source, full-featured
- Built-in reranking, filtering, hybrid search
- Cloud and self-hosted options

**4. Milvus**
- Open source, high performance
- Distributed, scalable
- Good for large-scale applications

**5. ChromaDB**
- Simple, lightweight, good for prototyping
- Easy API, in-memory or persistent

**6. Qdrant**
- High performance, filtering support
- Both open source and managed options

## 5.2 Embedding Models

**Sentence Transformers (Recommended for RAG):**
- all-MiniLM-L6-v2: 384 dims, fast, good quality
- all-mpnet-base-v2: 768 dims, better quality, slower
- multilingual-e5-base: 768 dims, supports 100+ languages

**OpenAI Embeddings:**
- text-embedding-3-small: 1536 dims
- text-embedding-3-large: 3072 dims
- State-of-the-art quality, costs money

**Cohere Embeddings:**
- embed-english-v3.0: High quality
- Supports search intent optimization

**Open Source (Self-hosted):**
- BGE (BAAI General Embeddings)
- E5 models (multilingual)

- Nomic Embed (open weights)

# 6. LLM INTEGRATION

## 6.1 LLM Options for RAG

**Cloud-Based (API):**

**OpenAI GPT-4:**
- Best quality, 128K context window
- Cost: $0.03-0.06 per 1K tokens

**Anthropic Claude 3.5:**
- Excellent quality, 200K context window
- Good for long document processing

**Google Gemini:**
- Competitive quality
- Good multimodal support

**Open Source (Self-hosted):**

**Llama 2/3 (Meta):**
- Free, open weights
- 70B model has good quality
- Needs GPU (requires 40GB VRAM for 70B)

**Mistral 7B:**
- Small, fast, good quality
- Can run on consumer GPUs

**Falcon:**
- Apache 2.0 license
- Good for commercial use

## 6.2 Context Window & Token Limits

**Important for RAG:**
- How much context can you pass to LLM?
- Typical formula: Query (50 tokens) + Retrieved docs (3000-5000 tokens) + Instructions (200 tokens)
- Need 5000-10000 token window for good RAG

**Context Windows:**
GPT-4: 128K tokens
Claude 3.5: 200K tokens
Llama 2: 4K tokens
Mistral: 32K tokens

**Token Counting:**
1 token ≈ 4 characters
1000 tokens ≈ 750 words
Use: tiktoken (OpenAI), transformers.tokenize

# 7. RAG PIPELINE IMPLEMENTATION

## 7.1 Basic RAG Implementation (Python)

**Step 1: Installation**
pip install langchain openai chromadb sentence-transformers python-dotenv

**Step 2: Load Documents & Create Index**

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma

# Load PDF
loader = PyPDFLoader("document.pdf")
documents = loader.load()

# Split into chunks
splitter = RecursiveCharacterTextSplitter(
chunk_size=1000,
chunk_overlap=200
)
chunks = splitter.split_documents(documents)

# Create embeddings and store
embeddings = HuggingFaceEmbeddings()
vectorstore = Chroma.from_documents(
chunks,
embeddings,
persist_directory="./chroma_db"
)
```

**Step 3: Create Retriever**

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
```

**Step 4: Build RAG Chain**

```
from langchain.llms import OpenAI
from langchain.chains import RetrievalQA

llm = OpenAI(model="gpt-3.5-turbo")

rag_chain = RetrievalQA.from_chain_type(
llm=llm,
chain_type="stuff",
retriever=retriever
)
```

**Step 5: Query**

```
result = rag_chain("What is machine learning?")
print(result['result'])
```

## 7.2 Advanced RAG with LangChain

**Custom Prompt Template:**

```
from langchain.prompts import PromptTemplate

template = 'Use following context to answer question. If unknown, say so.
Context: {context}
Question: {question}
Answer:'

prompt = PromptTemplate(template=template, input_variables=["context", "question"])
```

**Reranking Retrieved Results:**

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain.document_compressors import LLMChainExtractor

compressor = LLMChainExtractor.from_llm(llm, prompt)
compression_retriever = ContextualCompressionRetriever(
base_compressor=compressor,
base_retriever=retriever
)
```

**Multi-Query Retrieval:**

```
from langchain.retrievers.multi_query import MultiQueryRetriever

multi_query_retriever = MultiQueryRetriever.from_llm(
retriever=retriever,
llm=llm
)
```

# 8. POPULAR RAG FRAMEWORKS

## 8.1 LangChain

**What it is:** Python framework for building LLM applications
**Strengths:**
- Comprehensive abstractions for RAG
- Works with all major LLMs and vector DBs
- Great documentation
- Active community

**Weaknesses:**
- Sometimes opinionated design
- Can be slower than direct implementations

**Installation:** pip install langchain
**URL:** python.langchain.com

## 8.2 LlamaIndex (formerly GPT Index)

**What it is:** Data framework for LLM applications focused on indexing
**Strengths:**
- Specialized for RAG and indexing
- Better performance than LangChain for retrieval
- Simple API for document processing
- Built-in optimizations

**Weaknesses:**
- Smaller community than LangChain
- Fewer integrations

**Installation:** pip install llama-index
**URL:** gpt-index.readthedocs.io

## 8.3 Haystack

**What it is:** Production-ready NLP/RAG framework
**Strengths:**
- Built for production applications
- Flexible pipeline architecture
- Good documentation
- Handles complex workflows

**Weaknesses:**
- Steeper learning curve
- Less beginner-friendly

**Installation:** pip install haystack
**URL:** haystack.deepset.ai

## 8.4 Other Frameworks

**RAGStack:** Purpose-built for RAG (DataStax)
**Vectara:** Managed RAG platform

**Verba:** Open-source personal RAG system
**DSPy:** Programming language for RAG pipelines
**LiteLLM:** LLM abstraction layer

# 9. ADVANCED RAG TECHNIQUES

## 9.1 Chunking Strategies

**Fixed Size Chunking:**
- Split every N tokens
- Simple but may break sentences/context
- Good baseline: 512 tokens with 100 token overlap

**Semantic Chunking:**
- Split where semantic meaning changes
- Uses embedding similarity to find boundaries
- Better context preservation, slower

**Recursive Chunking:**
- Split by hierarchy: paragraph → sentence → token
- Default in LangChain, good balance

**Document-Aware Chunking:**
- Preserve document structure
- Different rules for headers, tables, code
- Best quality but complex

## 9.2 Query Optimization

**Query Expansion:**
- Generate multiple versions of query
- Retrieve with each version
- Union results

**Query Rewriting:**
- LLM rewrites unclear query
- Improves retrieval quality

**HyDE (Hypothetical Document Embeddings):**
- LLM generates hypothetical relevant document
- Use embedding for retrieval
- Better semantic matching

**Query Classification:**
- Route to different retrievers based on type
- Example: structural vs. semantic questions

## 9.3 Ranking & Reranking

**Cross-Encoder Reranking:**
- Use specialized model to score pairs
- More accurate than similarity scores
- Example: sentence-transformers cross-encoder
- Trade-off: Slower, better quality

**LLM-based Reranking:**
- Use LLM to judge relevance
- Most accurate but expensive

- Good for final ranking

**Diversity Ranking:**
- Avoid duplicate/redundant results
- Increase coverage of topics

**Fusion/Ensemble Ranking:**
- Combine scores from multiple retrievers
- RRF (Reciprocal Rank Fusion) algorithm

## 9.4 Advanced Techniques

**Knowledge Graph RAG:**
- Store entities and relationships
- Enable structured reasoning

**Multi-Modal RAG:**
- Retrieve both text and images
- Vision+Language models

**Routing & Dynamic Retrieval:**
- Route queries to different knowledge bases
- Use different retrieval strategies

**Iterative/Agentic RAG:**
- LLM decides what to retrieve next
- Multiple retrieval loops
- Enables complex reasoning

**Self-Adaptive RAG:**
- System learns optimal retrieval strategies
- Evaluate and improve over time

# 10. REAL-WORLD APPLICATIONS

## 10.1 Customer Support Chatbot

**Setup:**
- Knowledge base: All support docs, FAQs, product manuals
- Chunking: By product, topic, or FAQ
- Retrieval: Hybrid search (BM25 + semantic)
- LLM: Claude or GPT-4

**Benefits:**
- 24/7 support without human agents
- Accurate answers from company docs
- Can escalate to humans when needed
- Reduces support costs by 40-60%

## 10.2 Legal Document Analysis

**Setup:**
- Knowledge base: Legal documents, case law, precedents
- Chunking: By document type, clause type
- Retrieval: Semantic + BM25 hybrid
- LLM: GPT-4 (good reasoning)

**Benefits:**
- Review documents faster
- Identify relevant precedents
- Generate summaries and analysis
- Ensure compliance with regulations

## 10.3 Enterprise Search

**Setup:**
- Knowledge base: All company documents, wikis, policies
- Chunking: By document, section
- Retrieval: Semantic search with user permissions

**Benefits:**
- Employees find information faster
- Natural language search
- Source attribution
- Better knowledge sharing

## 10.4 Medical/Scientific Literature Search

**Setup:**
- Knowledge base: PubMed, medical literature
- Retrieval: Medical entity-aware search

**Benefits:**
- Find relevant research papers
- Summarize findings
- Identify contradictions

## 10.5 Code Documentation Assistant

**Setup:**
- Knowledge base: API docs, code examples
- Chunking: By class, function, module

**Benefits:**
- Auto-complete docs suggestions
- Answer API questions
- Generate code examples

# 11. EVALUATION METRICS

## 11.1 Retrieval Metrics

**Recall@K:** What % of relevant docs are in top K?
Formula: (# relevant docs in top K) / (total relevant docs)
Example: If 3 out of 5 relevant docs are in top 10, Recall@10 = 0.6

**Precision@K:** What % of top K are actually relevant?
Formula: (# relevant docs in top K) / K
Example: If 3 out of 10 retrieved docs are relevant, Precision@10 = 0.3

**MRR (Mean Reciprocal Rank):**
Average of 1 / (rank of first relevant doc)
Measures how high first relevant result is

**NDCG (Normalized Discounted Cumulative Gain):**
Considers ranking quality
Most relevant docs at top = higher score

**Hit Rate:** Did we find at least one relevant doc?
Binary: 1 if any relevant in top K, 0 otherwise

## 11.2 Generation Quality Metrics

**BLEU Score:**
Measures n-gram overlap with reference answer
Range: 0-1 (higher is better)
Issue: May miss semantically correct paraphrases

**ROUGE Score:**
Similar to BLEU, better for summaries
Measures recall of n-grams

**METEOR:**
Considers synonyms and paraphrases
Better than BLEU for evaluation

**BERTScore:**
Uses embeddings to measure semantic similarity
Better than BLEU/ROUGE for modern NLP

**Human Evaluation:**
Still most reliable
Evaluate: Relevance, Factuality, Readability, Completeness
Use multiple annotators, calculate inter-rater agreement

## 11.3 RAG-Specific Metrics

**Factual Consistency:**
Does the generated answer follow from context?
Can use entailment models to check

**Source Attribution Quality:**

Are citations correct and relevant?
Manual evaluation

**Latency & Cost:**
End-to-end response time
Cost per query (retrieval + LLM)

**Hallucination Rate:**
% of statements not supported by context
Measured by human evaluation

**Coverage:**
% of queries where system provides answer
vs. saying "I don't know"

# 12. COMPLETE STUDY ROADMAP

## PHASE 1: Foundations (1-2 weeks)

**Topics:**
✓ Understand LLM basics (transformers, tokens, context window)
✓ Vector embeddings and similarity (cosine, dot product)
✓ Information retrieval fundamentals (BM25, TF-IDF)
✓ Why RAG is needed (hallucinations, knowledge cutoff)

**Resources:**
• Andrew Ng: Gen AI for Everyone (DeepLearning.AI)
• Sebastian Raschka's LLM notes
• Understanding Transformers (YouTube)

**Practice:**
• Understand how embeddings work
• Calculate similarity between vectors

## PHASE 2: RAG Concepts (1-2 weeks)

**Topics:**
✓ RAG architecture overview
✓ Components: retrieval, ranking, generation
✓ Document chunking strategies
✓ Prompt engineering for RAG

**Resources:**
• "Retrieval-Augmented Generation for Knowledge-Intensive NLP" paper
• LangChain documentation (getting started)
• Vector database comparisons

**Practice:**
• Read RAG paper
• Understand different chunking approaches
• Experiment with different chunk sizes

## PHASE 3: Implementation (2-3 weeks)

**Topics:**
✓ Build basic RAG system with LangChain
✓ Set up vector database (ChromaDB, Pinecone)
✓ Load documents, chunk, embed
✓ Implement retrieval and generation

**Projects:**
1. Simple PDF Q&A; system
2. Custom knowledge base chatbot
3. Multi-document RAG system

**Resources:**
• LangChain cookbook
• LlamaIndex tutorials
• Vector database documentation

## PHASE 4: Advanced Techniques (2-3 weeks)

**Topics:**
✓ Reranking and ranking algorithms
✓ Query optimization techniques
✓ Hybrid retrieval (semantic + lexical)
✓ Evaluation metrics

**Projects:**
1. RAG system with reranking
2. Evaluate retrieval quality
3. Optimize chunking strategy

**Resources:**
• Cross-encoder papers
• Retrieval evaluation papers
• LlamaIndex advanced docs

## PHASE 5: Production & Optimization (2-3 weeks)

**Topics:**
✓ Caching and performance optimization
✓ Cost optimization (token usage)
✓ Monitoring and debugging
✓ Handling edge cases
✓ Security and access control

**Projects:**
1. Production RAG API
2. Cost-optimized system
3. Monitoring dashboard

**Resources:**
• LLM production patterns
• Caching strategies
• Cost optimization guides

## PHASE 6: Specialization (3-4 weeks)

**Choose one:**

A) **Agentic RAG:**
- Multi-step reasoning
- ReAct, Chain-of-Thought
- Dynamic retrieval

B) **Multi-Modal RAG:**
- Image + text retrieval
- Vision-language models

C) **Knowledge Graph RAG:**
- Structured knowledge
- Entity linking

D) **Domain-Specific RAG:**
- Specialized for: Legal, Medical, Code
- Domain-specific models/techniques

# 13. RESOURCES & REFERENCES

## 13.1 Research Papers (Must Read)

**Foundational RAG Paper:**
"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"
Lewis et al. (2020) - Facebook AI
URL: arxiv.org/abs/2005.11401
Key insight: First major RAG paper, combines retrieval with seq2seq

**Dense Retrieval:**
"Dense Passage Retrieval for Open-Domain Question Answering"
Karpukhin et al. (2020)
URL: arxiv.org/abs/2004.04906

**Query Optimization:**
"Precise Zero-Shot Dense Retrieval without Relevance Labels"
Gao et al. (2023) - HyDE
URL: arxiv.org/abs/2212.10496

**Cross-Encoders for Ranking:**
"Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks"
Reimers and Gurevych (2019)

**Advanced RAG:**
"Self-RAG: Learning to Retrieve, Generate, and Critique"
Asai et al. (2023)
URL: arxiv.org/abs/2310.11511

## 13.2 Online Courses & Tutorials

**Free Resources:**
• DeepLearning.AI: "Retrieval Augmented Generation" course
• LangChain YouTube tutorials
• LlamaIndex documentation and guides
• Hugging Face RAG tutorials

**Paid Courses:**
• Coursera: LLM courses (various instructors)
• DataCamp: Advanced NLP courses

**YouTube Channels:**
• Sam Witteveen's LangChain series
• Jeremy Howard's Fast.ai
• Full Stack Deep Learning

## 13.3 Tools & Libraries

**Framework & Tools:**
• LangChain: python.langchain.com
• LlamaIndex: gpt-index.readthedocs.io
• Haystack: haystack.deepset.ai
• RAGStack: datastax.com/products/ragstack

**Vector Databases:**
• Pinecone: pinecone.io
• Weaviate: weaviate.io
• Milvus: milvus.io
• ChromaDB: trychroma.com

**Embedding Models:**
• Hugging Face Models: huggingface.co/models
• OpenAI API: openai.com
• Cohere: cohere.com

**LLM Providers:**
• OpenAI: openai.com
• Anthropic (Claude): anthropic.com
• Google (Gemini): deepmind.google
• Together AI: together.ai

## 13.4 Benchmark Datasets

**For Testing RAG:**
• SQuAD: Stanford Question Answering Dataset
• MS MARCO: Large-scale Information Ranking
• Natural Questions: Real Google search queries
• WebQuestions: Web-based QA
• TriviaQA: Large-scale QA dataset

**For Specific Domains:**
• SciBENCH: Scientific papers
• LEGAL-BERT: Legal documents
• MedQA: Medical questions
• CodeSearch: Code search

# SUCCESS TIPS FOR RAG DEVELOPMENT

**1. Start Simple:**
Begin with basic RAG before adding complexity. Get retrieval working before optimization.

**2. Evaluate Early:**
Don't build for weeks without measuring. Create test set early, evaluate frequently.

**3. Focus on Retrieval:**
Quality answer requires quality retrieval. Spending 80% of effort on retrieval pays off.

**4. Chunk Wisely:**
Chunk size and strategy matter most. Experiment: 512, 1024, 2048 tokens with overlap.

**5. Optimize Costs:**
LLM calls are expensive. Use caching, batch processing, smaller models where possible.

**6. Handle Edge Cases:**
What if no relevant docs found? What if query is off-topic? Plan for these cases.

**7. Monitor Production:**
Log queries, responses, and user feedback. Use for continuous improvement.

**8. Security First:**
Control who can access which documents. Implement proper authentication and authorization.

**9. User Feedback Loop:**
Collect feedback on answer quality. Use to identify weak areas.

**10. Stay Updated:**
RAG is evolving fast. Follow papers, framework updates, new techniques monthly.

# COMMON RAG CHALLENGES & SOLUTIONS

**Challenge 1: Poor Retrieval Quality**
Symptoms: Getting irrelevant documents
Solutions:
• Better chunking strategy
• Improve embedding model
• Use hybrid retrieval (semantic + BM25)
• Add reranking with cross-encoders

**Challenge 2: Lost Context Between Retrieval and Generation**
Symptoms: Retrieved docs not used effectively in answer
Solutions:
• Better prompt engineering
• Use document summaries as context
• Implement reranking
• Reduce number of retrieved docs

**Challenge 3: Hallucinations Still Occurring**
Symptoms: Model invents information not in context
Solutions:
• Use smaller, more factual models
• Add fact-checking step
• Force citations from context
• Use Claude or GPT-4 (more reliable)

**Challenge 4: Slow Response Times**
Symptoms: Takes 10+ seconds per query
Solutions:
• Cache embeddings
• Use async retrieval
• Reduce number of docs to rerank
• Use faster embedding models
• Optimize LLM calls (streaming)

**Challenge 5: High Costs**
Symptoms: Expensive API calls
Solutions:
• Use smaller, faster models
• Cache common queries
• Retrieve fewer documents
• Use open-source models self-hosted
• Optimize token usage